

Name: Sneha Gopal Bhalekar Roll_NO: cs05

Aim : To implement a Bagging (Bootstrap Aggregating) ensemble classifier from scratch using Python, apply it to the Pima Indians Diabetes Database, and evaluate its performance against a single base classifier (Decision Tree)**.

Objectives

1. Understand Ensemble Learning Principles: Comprehend how combining the predictions of multiple diverse models reduces overall model variance.
2. Master Bootstrapping: Implement random resampling of data with replacement from scratch to generate distinct training subsets.
3. Develop a Voting System: Build an aggregation mechanism (Majority Voting) from scratch to consolidate predictions.
4. Learn Baseline Evaluation: Compare the performance of a high-variance, low-bias single decision tree against the more stable Bagging ensemble.
5. Analyze Performance Metrics: Calculate and visualize classification metrics, including a Confusion Matrix, Accuracy, Precision, Recall, and ROC-AUC curve.

Relevant Theory Ensemble Methods and Bagging Ensemble learning combines several individual classifiers (base learners) to produce a unified model that generalizes better than any single model on unseen data. Bootstrap Aggregating (Bagging) is a parallel ensemble method designed primarily to reduce variance. High-variance models—such as deep, fully-grown decision trees—are highly sensitive to small changes in their training datasets. Bagging addresses this by smoothing out individual model errors without increasing bias.

The two core components of Bagging are:

1. **Bootstrapping (Sampling with Replacement):** For a training set of size N , we generate B new datasets of the same size N . Each instance is selected at random from the original dataset with replacement. This means a single instance can be selected multiple times, while on average, only about 63.2% of the unique training samples are captured in each bootstrap, leaving the rest as out-of-bag (OOB) samples.
2. **Aggregating (Majority Voting):** A separate base learner $h_b(x)$ is trained independently on each bootstrap sample. For classification, predictions are aggregated by passing a test sample to all B models and taking a majority vote:

$$\hat{G}_{\text{bag}}(x) = \underset{k}{\operatorname{argmax}} \sum_{b=1}^B I(h_b(x) = k)$$

An alternative to standard Bagging is subbagging (subsample aggregating), where smaller training subsets (often $N/2$) are drawn without replacement to reduce correlation and accelerate computation.

Software Requirements

- **Language:** Python 3.x
- **Core Libraries:**
 - **NumPy** for matrix manipulation and custom bootstrapping operations.
 - **Pandas** for loading, indexing, and cleaning tabular data.
 - **Matplotlib / Seaborn** for exploratory scatter plots, bar charts, and performance visualization.
 - **Scikit-learn** for train-test splitting, training individual base decision trees, and comparison with built-in Bagging algorithms.

Dataset Information

The experiment utilizes the **Pima Indians Diabetes Database** from the UCI Machine Learning Repository. The dataset contains medical details for 768 female Pima Indian patients living in Arizona, categorized as either diabetic or non-diabetic.

- **Total Instances:** 768
- **Features (8 predictor variables):**
 1. **Pregnancies**: Number of times pregnant.
 2. **Glucose**: Plasma glucose concentration over 2 hours in an oral glucose tolerance test.
 3. **BloodPressure**: Diastolic blood pressure (mm Hg).
 4. **SkinThickness**: Triceps skin fold thickness (mm).
 5. **Insulin**: 2-Hour serum insulin ($\mu\text{U/ml}$).
 6. **BMI**: Body mass index (weight in kg/m^2).
 7. **DiabetesPedigreeFunction**: A function representing genetic predisposition.
 8. **Age**: Age of the subject in years.
- **Target Label (9th column):** **Outcome**
 - 0 represents non-diabetic.
 - 1 represents diabetic.

Experimental Tasks

Task 1: Environment Setup and Dataset Acquisition

Use dataset using the link

<https://www.kaggle.com/datasets/jamaltariqcheema/pima-indians-diabetes-dataset>

Goal: Set up the workspace and load the data.

Instructions:

1. Import the necessary modules: **numpy**, **pandas**, **matplotlib.pyplot**, and **sklearn**.
2. Load the file **pima-indians-diabetes.data** into a Pandas DataFrame, assigning appropriate column headers.
3. Print the following:
 - First five records of the dataset.
 - Dataset shape, expected to be 768×9 .
 - Data types of all features.
 - Descriptive statistics of the numeric features.

```
# TASK 1: Environment Setup and Dataset Acquisition

# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn

# Load dataset
diabetes = pd.read_csv('/content/diabetes.csv')

# First 5 records
print("First 5 records:")
display(diabetes.head())
```

```
# Dataset shape
print("\nDataset Shape:")
print(diabetes.shape)

# Data types
print("\nData Types:")
print(diabetes.dtypes)

# Descriptive statistics
print("\nDescriptive Statistics:")
display(diabetes.describe())
```

First 5 records:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72.0	35	169.5	33.6	0.627	50	1
1	1	85	66.0	29	102.5	26.6	0.351	31	0
2	8	183	64.0	32	169.5	23.3	0.672	32	1
3	1	89	66.0	23	94.0	28.1	0.167	21	0
4	0	137	40.0	35	168.0	43.1	2.288	33	1

Dataset Shape:
(768, 9)

Data Types:

Pregnancies	int64
Glucose	int64
BloodPressure	float64
SkinThickness	int64
Insulin	float64
BMI	float64
DiabetesPedigreeFunction	float64
Age	int64
Outcome	int64

dtype: object

Descriptive Statistics:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000		768.000000
mean	3.845052	121.677083	72.389323	29.089844	141.753906	32.434635		0.471876
std	3.369578	30.464161	12.106039	8.890820	89.100847	6.880498		0.331329
min	0.000000	44.000000	24.000000	7.000000	14.000000	18.200000		0.078000
25%	1.000000	99.750000	64.000000	25.000000	102.500000	27.500000		0.243750
50%	3.000000	117.000000	72.000000	28.000000	102.500000	32.050000		0.372500
75%	6.000000	140.250000	80.000000	32.000000	169.500000	36.600000		0.626250
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000		2.420000

✓ Task 2: Exploratory Data Analysis (EDA)

Goal: Visually inspect feature distributions and class balance.

Instructions:

1. Calculate the **absolute count** and **percentage** of diabetic (`Outcome = 1`) and non-diabetic (`Outcome = 0`) cases. Plot the class balance using a **bar chart**.

2. Create a **scatter plot** of **Age** (Feature 7) against **Glucose** (Feature 1), color-coded according to the target variable **Outcome**.
3. Briefly explain why a simple linear decision boundary is insufficient to clearly separate the diabetic and non-diabetic classes.

```
# TASK 2: Exploratory Data Analysis (EDA)

# 1. Calculate class counts
class_counts = diabetes['Outcome'].value_counts().sort_index()

print("Class Counts:")
print("Non-Diabetic (0):", class_counts[0])
print("Diabetic (1):", class_counts[1])

# 2. Calculate class percentages
class_percentages = diabetes['Outcome'].value_counts(
    normalize=True
).sort_index() * 100

print("\nClass Percentages:")
print("Non-Diabetic (0):", class_percentages[0], "%")
print("Diabetic (1):", class_percentages[1], "%")

# 3. Plot class balance
labels = ['Non-Diabetic', 'Diabetic']
counts = [class_counts[0], class_counts[1]]

plt.bar(labels, counts)

plt.xlabel('Outcome')
plt.ylabel('Number of Patients')
plt.title('Class Balance of Diabetes Dataset')

plt.show()

# 4. Scatter plot: Age vs Glucose

plt.scatter(
    diabetes[diabetes['Outcome'] == 0]['Age'],
    diabetes[diabetes['Outcome'] == 0]['Glucose'],
    label='Non-Diabetic'
)

plt.scatter(
    diabetes[diabetes['Outcome'] == 1]['Age'],
    diabetes[diabetes['Outcome'] == 1]['Glucose'],
    label='Diabetic'
)

plt.xlabel('Age')
plt.ylabel('Glucose')
plt.title('Age vs Glucose by Diabetes Outcome')
plt.legend()

plt.show()
```

Class Counts:

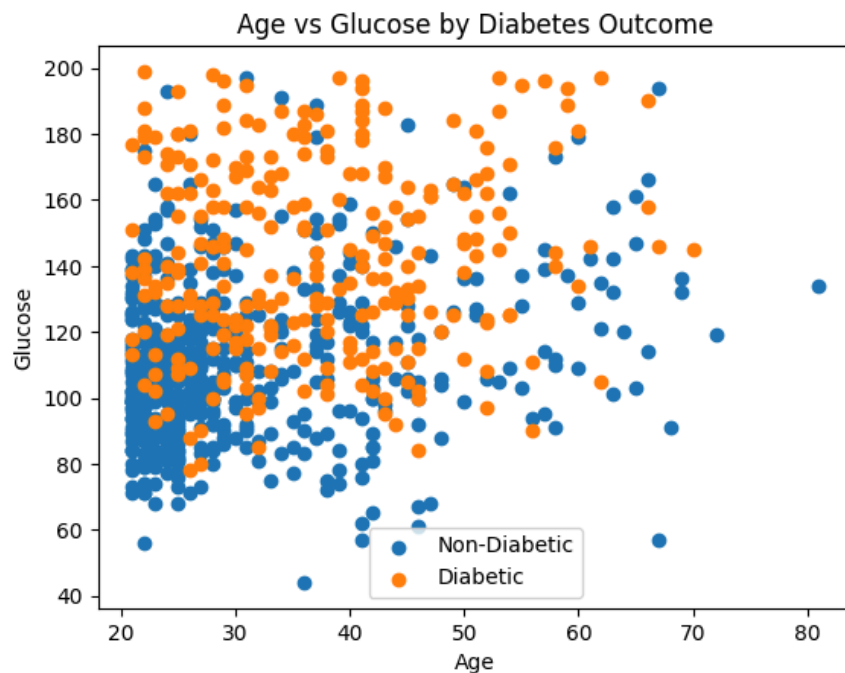
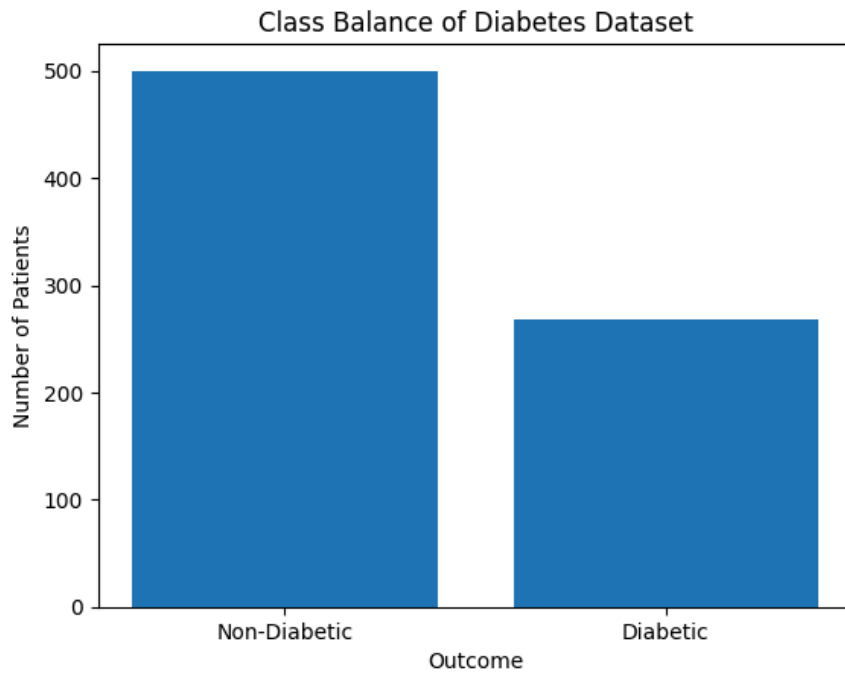
Non-Diabetic (0): 500

Diabetic (1): 268

Class Percentages:

Non-Diabetic (0): 65.10416666666666 %

Diabetic (1): 34.89583333333333 %



Task 3: Preprocessing and Handling Missing Values

Goal: Handle anomalies and prepare numerical features for further analysis.

Instructions:

1. Identify the features where a value of (0) is physically impossible: [$\{\text{Glucose}, \text{BloodPressure}, \text{SkinThickness}, \text{Insulin}, \text{BMI}\}$]
2. Replace all invalid (0) values in these features with (NaN).

3. Handle the resulting missing values by:

- imputing them using the **mean** or **median** of the respective feature, or
- dropping rows containing missing values to maintain numerical consistency.

```
# TASK 3: Preprocessing and Handling Missing Values

# Features where 0 is physically impossible
zero_invalid_features = [
    'Glucose',
    'BloodPressure',
    'SkinThickness',
    'Insulin',
    'BMI'
]

# 1. Count zero values
print("Zero values before preprocessing:")

for column in zero_invalid_features:
    print(column, ":", (diabetes[column] == 0).sum())

# 2. Replace 0 with NaN
diabetes[zero_invalid_features] = diabetes[
    zero_invalid_features
].replace(0, np.nan)

# 3. Check missing values
print("\nMissing values after replacing 0 with NaN:")
print(diabetes.isnull().sum())

# 4. Impute missing values using median
for column in zero_invalid_features:
    diabetes[column] = diabetes[column].fillna(
        diabetes[column].median()
    )

# 5. Verify
print("\nMissing values after median imputation:")
print(diabetes.isnull().sum())
```

Zero values before preprocessing:

```
Glucose : 0
BloodPressure : 0
SkinThickness : 0
Insulin : 0
BMI : 0
```

Missing values after replacing 0 with NaN:

```
Pregnancies      0
Glucose          0
BloodPressure     0
SkinThickness     0
Insulin          0
BMI              0
DiabetesPedigreeFunction  0
Age              0
Outcome          0
dtype: int64
```

Missing values after median imputation:

```
Pregnancies      0
Glucose          0
BloodPressure     0
SkinThickness     0
```

```

Insulin      0
BMI          0
DiabetesPedigreeFunction  0
Age          0
Outcome      0
dtype: int64

```

✓ Task 4: Stratified Train-Test Splitting

Goal: Divide the data while preserving class proportions.

Instructions:

1. Isolate the feature matrix (X) and the target vector (y): [$X = \text{first 8 columns}$] [$y = \text{9th column}$]
2. Split the dataset into: [$\text{Training Data} = 80\%$] [$\text{Test Data} = 20\%$]

Use **stratified sampling** to ensure that the class proportions in the target variable (y) are preserved in both the training and test datasets.

```

# TASK 4: Stratified Train-Test Splitting

from sklearn.model_selection import train_test_split

# Separate features and target
X = diabetes.drop('Outcome', axis=1)
y = diabetes['Outcome']

# Split into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Display shapes
print("Training data shape:")
print(X_train.shape)

print("\nTesting data shape:")
print(X_test.shape)

print("\nTraining target shape:")
print(y_train.shape)

print("\nTesting target shape:")
print(y_test.shape)

# Check class distribution
print("\nTraining class distribution:")
print(y_train.value_counts(normalize=True) * 100)

print("\nTesting class distribution:")
print(y_test.value_counts(normalize=True) * 100)

```

```

Training data shape:
(614, 8)

```

```

Testing data shape:
(154, 8)

```

```

Training target shape:
(614,)

```

```

Testing target shape:
(154,)

```

```

Training class distribution:
Outcome
0    65.14658
1    34.85342
Name: proportion, dtype: float64

Testing class distribution:
Outcome
0    64.935065
1    35.064935
Name: proportion, dtype: float64

```

✓ Task 5: Custom Bootstrap Sampling Function

Goal: Implement the sampling-with-replacement mechanism.

Instructions:

1. Write a custom Python function (`get_bootstrap_sample(X, y)`) using NumPy's random generation tools, such as: `[ np.random.choice \quad \text{or} \quad np.random.randint ]`
2. The function must return a resampled feature matrix ($X^\wedge$) *and target vector* ($y^\wedge$) of the same size (N) as the training set: `[ |X^wedge| = |y^wedge| = N ]`

Sampling should be performed **with replacement**, allowing duplicate rows to appear in the bootstrap sample.

3. Verify that the function works correctly by demonstrating that:
 - Some original row indices are **duplicated** in the bootstrap sample.
 - Some original row indices are **omitted**, forming the **Out-of-Bag (OOB)** samples.

```

# TASK 5: Custom Bootstrap Sampling Function

# Create bootstrap sampling function
def get_bootstrap_sample(X, y):

    # Number of training samples
    N = len(X)

    # Random sampling with replacement
    indices = np.random.choice(
        N,
        size=N,
        replace=True
    )

    # Create bootstrap dataset
    X_bootstrap = X.iloc[indices]
    y_bootstrap = y.iloc[indices]

    return X_bootstrap, y_bootstrap, indices

# Generate one bootstrap sample
X_bootstrap, y_bootstrap, sample_indices = get_bootstrap_sample(
    X_train,
    y_train
)

# Check sample sizes
print("Original training data size:")
print(len(X_train))

print("\nBootstrap sample size:")
print(len(X_bootstrap))

```

```

# Check duplicated indices
unique_indices, counts = np.unique(
    sample_indices,
    return_counts=True
)

duplicated_indices = unique_indices[counts > 1]

print("\nNumber of duplicated original indices:")
print(len(duplicated_indices))

print("\nSome duplicated indices:")
print(duplicated_indices[:10])

# Find OOB samples
all_indices = np.arange(len(X_train))

oob_indices = np.setdiff1d(
    all_indices,
    np.unique(sample_indices)
)

print("\nNumber of OOB samples:")
print(len(oob_indices))

print("\nSome OOB indices:")
print(oob_indices[:10])

```

Original training data size:
614

Bootstrap sample size:
614

Number of duplicated original indices:
159

Some duplicated indices:
[0 3 4 7 10 16 18 32 38 41]

Number of OOB samples:
221

Some OOB indices:
[5 6 8 21 22 24 25 26 27 28]

✓ Task 6: Growing the Ensemble of Base Classifiers

Goal: Programmatically train diverse base learners.

Instructions:

1. Initialize an empty list to store the trained models and choose the number of bootstrap iterations, for example: [B = 50]
2. Execute the following steps for (B) iterations:
 - Generate a unique bootstrap training dataset: [(X^{*b}, y^{*b})] using the bootstrap sampling function from **Task 5**.
 - Instantiate a Scikit-Learn `DecisionTreeClassifier` with: [\texttt{max_depth=None}] This ensures that the decision trees are **fully grown**, resulting in high-variance base learners.
 - Fit the decision tree on the bootstrapped training subset: [\texttt{Model}_b.fit(X^{*b}, y^{*b})]

- Append the trained model to the ensemble list.
3. After completing all (B) iterations, the ensemble should contain: [B = 50] independently trained decision trees.

```
# TASK 6: Growing the Ensemble of Base Classifiers

from sklearn.tree import DecisionTreeClassifier

# Number of bootstrap iterations / trees
B = 50

# Empty list to store trained models
ensemble = []

# Train 50 decision trees
for b in range(B):

    # Generate a new bootstrap training dataset
    X_bootstrap, y_bootstrap, sample_indices = get_bootstrap_sample(
        X_train,
        y_train
    )

    # Create fully grown Decision Tree
    model = DecisionTreeClassifier(
        max_depth=None
    )

    # Fit the model
    model.fit(X_bootstrap, y_bootstrap)

    # Add trained model to ensemble
    ensemble.append(model)

# Verify
print("Number of trees in the ensemble:", len(ensemble))

Number of trees in the ensemble: 50
```

✓ Task 7: Implementing Aggregation (Majority Voting) from Scratch

Goal: Consolidate individual model decisions using majority voting.

Instructions:

- Write a custom function: [$\text{custom_bagging_predict}(\text{ensemble}, X_{\text{test}})$] to generate the final predictions.
- For each sample in (X_{test}) , collect the predictions generated by all (B) decision trees in the ensemble: [$\hat{y}_1, \hat{y}_2, \dots, \hat{y}_B$]
- Apply a **majority voting** rule to determine the final predicted class. The class with the highest number of votes is selected: [$\hat{y} = \underset{c \in \{0,1\}}{\arg\max} \sum_{b=1}^B I(\hat{y}_b = c)$]
where $I(\cdot)$ is an indicator function.
- The final prediction for each test sample must therefore be either: [$\hat{y} \in \{0,1\}$]
- The function should return the final majority-voted predictions for all samples in (X_{test}) .

```
# TASK 7: Implementing Aggregation (Majority Voting)

def custom_bagging_predict(ensemble, X_test):

    # Store predictions from all decision trees
    all_predictions = []
```

```

# Get predictions from each tree
for model in ensemble:
    predictions = model.predict(X_test)
    all_predictions.append(predictions)

# Convert to NumPy array
all_predictions = np.array(all_predictions)

# Apply majority voting
final_predictions = []

for i in range(len(X_test)):

    votes_for_0 = np.sum(all_predictions[:, i] == 0)
    votes_for_1 = np.sum(all_predictions[:, i] == 1)

    if votes_for_1 > votes_for_0:
        final_predictions.append(1)
    else:
        final_predictions.append(0)

return np.array(final_predictions)

# Generate final Bagging predictions
y_pred_bagging = custom_bagging_predict(
    ensemble,
    X_test
)

# Display results
print("Number of test samples:", len(X_test))
print("Number of predictions:", len(y_pred_bagging))

print("\nUnique predicted classes:")
print(np.unique(y_pred_bagging))

print("\nFirst 20 Bagging Predictions:")
print(y_pred_bagging[:20])

Number of test samples: 154
Number of predictions: 154

Unique predicted classes:
[0 1]

First 20 Bagging Predictions:
[0 0 0 1 0 0 1 1 0 1 0 1 0 0 1 1 0 0 1 0]

```

Task 8: Single Classifier vs. Bagging Performance Comparison

- **Goal:** Observe the variance-reduction benefits of ensembling.
- **Instructions:**
 1. Train a standalone, single (`DecisionTreeClassifier`) (fully grown, no depth limit) on the original, un-resampled training data.
 2. Predict the classes for (`X_test`) using both the single tree and your custom Bagging classifier.
 3. Construct a **Confusion Matrix** for both models.
 4. Compute and print **Accuracy, Precision, Recall, and F1-Score** for both models, demonstrating how Bagging stabilizes results and reduces error rates.

```
# TASK 8: Single Classifier vs. Bagging Performance Comparison
```

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

# -----
# 1. Train a single fully grown Decision Tree
# -----

single_tree = DecisionTreeClassifier(
    max_depth=None,
    random_state=42
)

single_tree.fit(X_train, y_train)

# Predict using single tree
y_pred_single = single_tree.predict(X_test)

# -----
# 2. Predict using Custom Bagging
# -----

y_pred_bagging = custom_bagging_predict(
    ensemble,
    X_test
)

# -----
# 3. Calculate metrics for Single Decision Tree
# -----

single_accuracy = accuracy_score(y_test, y_pred_single)
single_precision = precision_score(y_test, y_pred_single)
single_recall = recall_score(y_test, y_pred_single)
single_f1 = f1_score(y_test, y_pred_single)

# -----
# 4. Calculate metrics for Bagging
# -----

bagging_accuracy = accuracy_score(y_test, y_pred_bagging)
bagging_precision = precision_score(y_test, y_pred_bagging)
bagging_recall = recall_score(y_test, y_pred_bagging)
bagging_f1 = f1_score(y_test, y_pred_bagging)

# -----
# 5. Display Performance
# -----

print("===== SINGLE DECISION TREE =====")
print("Accuracy :", single_accuracy)
print("Precision:", single_precision)
print("Recall   :", single_recall)
print("F1-Score :", single_f1)

print("\n===== CUSTOM BAGGING =====")
print("Accuracy :", bagging_accuracy)

```

```

print("Precision:", bagging_precision)
print("Recall  :", bagging_recall)
print("F1-Score :", bagging_f1)

# -----
# 6. Confusion Matrices
# -----

cm_single = confusion_matrix(y_test, y_pred_single)
cm_bagging = confusion_matrix(y_test, y_pred_bagging)

print("\nConfusion Matrix - Single Decision Tree:")
print(cm_single)

print("\nConfusion Matrix - Custom Bagging:")
print(cm_bagging)

```

```

===== SINGLE DECISION TREE =====
Accuracy : 0.8116883116883117
Precision: 0.7450980392156863
Recall   : 0.7037037037037037
F1-Score : 0.7238095238095238

```

```

===== CUSTOM BAGGING =====
Accuracy : 0.8896103896103896
Precision: 0.8490566037735849
Recall   : 0.8333333333333334
F1-Score : 0.8411214953271028

```

```

Confusion Matrix - Single Decision Tree:
[[87 13]
 [16 38]]

```

```

Confusion Matrix - Custom Bagging:
[[92  8]
 [ 9 45]]

```

Task 9: ROC-AUC Analysis and Curve Visualization

- **Goal:** Measure classification performance at different thresholds.
- **Instructions:**
 1. Calculate predicted class probabilities by averaging the probability estimates (`predict_proba`) across all (B) decision trees in your custom Bagging model.
 2. Plot the **Receiver Operating Characteristic (ROC) curve** for both the single decision tree and the custom Bagging model on a single chart.
 3. Compute and contrast the **Area Under the Curve (AUC)** for both configurations, noting the improvement.

```

# TASK 9: ROC-AUC Analysis and Curve Visualization

from sklearn.metrics import roc_curve, roc_auc_score

# -----
# 1. Single Decision Tree probabilities
# -----

y_prob_single = single_tree.predict_proba(X_test)[: , 1]

# -----
# 2. Bagging probabilities
# -----

all_probabilities = []

```

```

for model in ensemble:
    probabilities = model.predict_proba(X_test)[:, 1]
    all_probabilities.append(probabilities)

all_probabilities = np.array(all_probabilities)

# Average probabilities from all 50 trees
y_prob_bagging = np.mean(all_probabilities, axis=0)

# -----
# 3. ROC Curves
# -----

fpr_single, tpr_single, _ = roc_curve(
    y_test,
    y_prob_single
)

fpr_bagging, tpr_bagging, _ = roc_curve(
    y_test,
    y_prob_bagging
)

# -----
# 4. AUC
# -----

auc_single = roc_auc_score(
    y_test,
    y_prob_single
)

auc_bagging = roc_auc_score(
    y_test,
    y_prob_bagging
)

# -----
# 5. Print AUC
# -----

print("AUC - Single Decision Tree:", auc_single)
print("AUC - Custom Bagging:", auc_bagging)

# -----
# 6. Plot ROC Curves
# -----

plt.figure(figsize=(8, 6))

plt.plot(
    fpr_single,
    tpr_single,
    label=f'Single Decision Tree (AUC = {auc_single:.3f})'
)

plt.plot(
    fpr_bagging,
    tpr_bagging,
    label=f'Custom Bagging (AUC = {auc_bagging:.3f})'
)

plt.plot(
    [0, 1],

```

```

[0, 1],
linestyle='--',
label='Random Classifier'
)

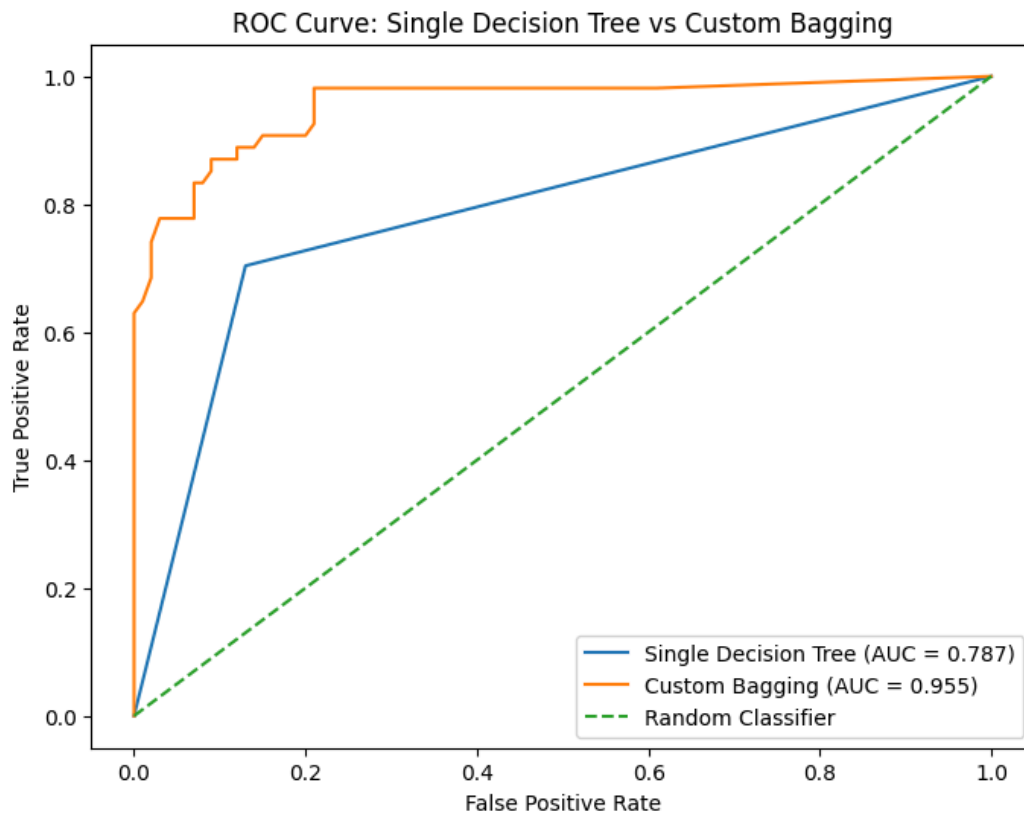
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve: Single Decision Tree vs Custom Bagging')
plt.legend()

plt.show()

```

AUC - Single Decision Tree: 0.7868518518518519

AUC - Custom Bagging: 0.9549074074074073



Task 10: Validation against Scikit-Learn's BaggingClassifier

- **Goal:** Verify custom code correctness.
- **Instructions:**
 1. Instantiate Scikit-Learn's built-in (`BaggingClassifier`) using (`DecisionTreeClassifier(max_depth=None)`) as the base estimator and (`n_estimators=50`).
 2. Fit the classifier on the training set and evaluate it on the test set.
 3. Compare the resulting test accuracy and confusion matrix against your custom implementation from **Task 7** and **Task 8** to validate your custom bootstrap and aggregation logic.

```
# TASK 10: Validation against Scikit-Learn's BaggingClassifier
```

```

from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

```

```

# 1. Create base Decision Tree
base_tree = DecisionTreeClassifier(

```

```
        max_depth=None,
        random_state=42
    )

# 2. Create Scikit-Learn Bagging Classifier
sklearn_bagging = BaggingClassifier(
    estimator=base_tree,
    n_estimators=50,
    random_state=42
)

# 3. Train the classifier
sklearn_bagging.fit(X_train, y_train)

# 4. Predict on test data
y_pred_sklearn = sklearn_bagging.predict(X_test)

# 5. Calculate accuracy
sklearn_accuracy = accuracy_score(
    y_test,
    y_pred_sklearn
)

# 6. Calculate confusion matrix
sklearn_cm = confusion_matrix(
    y_test,
    y_pred_sklearn
)

# 7. Custom Bagging accuracy
custom_accuracy = accuracy_score(
    y_test,
    y_pred_bagging
)

# 8. Display results
print("===== CUSTOM BAGGING =====")

print("Accuracy:")
print(custom_accuracy)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_bagging))

print("\n===== SCIKIT-LEARN BAGGING =====")

print("Accuracy:")
print(sklearn_accuracy)

print("\nConfusion Matrix:")
print(sklearn_cm)

# 9. Accuracy comparison
print("\n===== COMPARISON =====")

print("Custom Bagging Accuracy      :", custom_accuracy)
print("Scikit-Learn Bagging Accuracy:", sklearn_accuracy)
```

```
===== CUSTOM BAGGING =====  
Accuracy:  
0.8896103896103896  
  
Confusion Matrix:  
[[92  8]  
 [ 9 45]]  
  
===== SCIKIT-LEARN BAGGING =====  
Accuracy:  
0.8831168831168831  
  
Confusion Matrix:  
[[91  9]  
 [ 9 45]]  
  
===== COMPARISON =====  
Custom Bagging Accuracy      : 0.8896103896103896  
Scikit-Learn Bagging Accuracy: 0.8831168831168831
```

Conclusion and Interpretation

Students should summarize their findings by answering the following key questions:

1. **Variance and Stability:** How does the test accuracy of the Bagging ensemble compare to the single decision tree? Why does Bagging provide more consistent performance across different train-test splits?
2. **Wisdom of Crowds:** How does random resampling with replacement (bootstrapping) foster model diversity, and how does the consensus voting rule translate to a "Wisdom of Crowds" effect?
3. **Clinical Interpretation:** In predicting diabetes, why are False Negatives (predicting non-diabetic for a diabetic patient) especially costly, and how does Bagging's Recall metric help minimize this risk compared to a single decision tree?